Data Structures Project-1 Report

Drop Number Game with Multi-Linked List

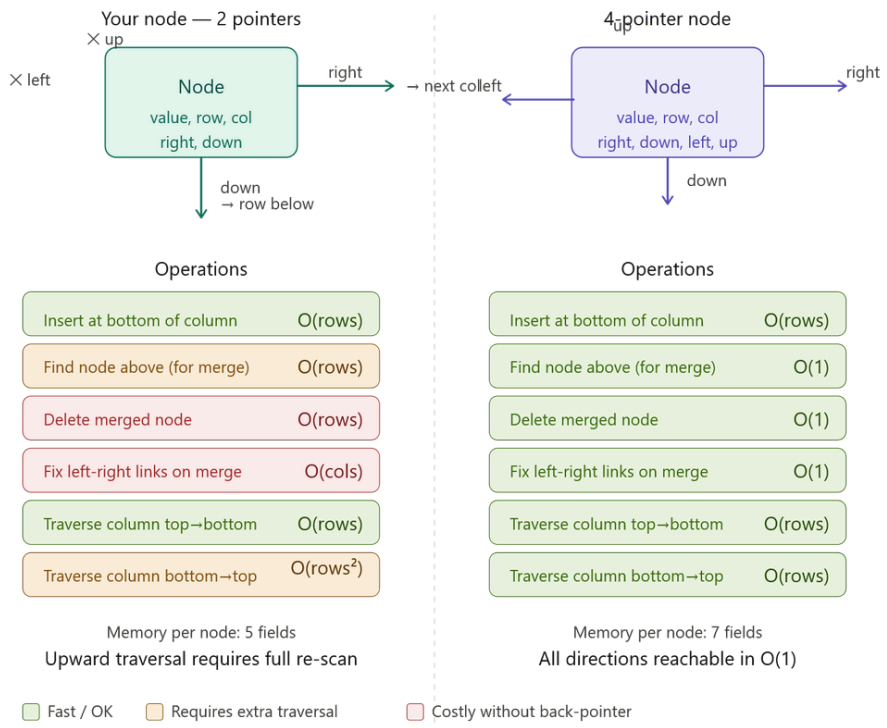
Roaa Alsham - 2421051350

1. Introduction

- **Goal:** The objective of the game is to merge falling tiles of the same number to double their value, continuously creating larger numbers until the grid fills up.
- **Constraints:** The core challenge of this project was implementing the game mechanics strictly using a multi-linked list. As per the project requirements, the use of arrays or matrices to store the 7x5 grid data was strictly prohibited.

2. Data Structure Design

- **The Node Class:**
  - **Structure:** Each Node stores data (value, row, col) and uses four directional pointers: up, down, left, and right.
  - **Node Design Decision:** To determine which node design is the best for a game that depends on merge() function and requires continuous traversal, I compared the time complexity between two designs accordingly a 4-pointer node was chosen over a 2-pointer (right, down) node.



- **The Grid Architecture & The head Pointer:**
  - To optimize memory, the grid does not store empty cells.
  - The head pointer points to the very first node dropped. Every other node in the grid is reachable by first navigating to absolute bottom-left via the head pointer.

3. Implementation Details (Updating the Linked List)

A. Vertical Navigation Because blocks always stack upwards from the bottom of the grid, navigation relies on finding the bottom-most block in a column.

- getColumnFloor(int col): This method traverses down and left from the head to find the absolute bottom-left node of the entire grid. From there, it walks right across the horizontal "floor" links until it finds the target column.

Complexity : O(R+C)

- getColumnPeak(int col): To find where a new block should land, the program first locates the column's floor, then traverses the up pointers until it reaches the highest stacked block (the peak).

Complexity : O(R+C)

B. Dropping a Block (dropBlock) When a block is dropped into a target column, the program checks the peak of that column:

- **Insert Path:** If the column is empty, the block lands at the maximum depth (ROWS - 1). If the column has blocks but the top block does not match the incoming value, the new block is stacked exactly one row above the current peak (peak.row - 1).
- **Merge Path:** If the incoming block's value matches the peak block's value, a new node is not created. Instead, the existing peak's value is multiplied by 2 in place, and the cascadeMerge method is called recursively to check any other merge operations.

Complexity**:** O(C×(R+C))

C. Horizontal Wiring (wireHorizontal) When a new node is inserted, it must connect to its adjacent neighbors on the exact same row to maintain horizontal integrity.

- The method scans the columns to the immediate left and right.
- It finds the peak of those adjacent columns and walks down until it finds a node whose row exactly matches the new node's row.
- Once found, the left and right pointers of both nodes are doubly linked.

Complexity**:** O(C×(R+C))

D. Cascade Merging Logic (cascadeMerge) In the standard Drop Number game, a newly merged block can trigger a chain reaction if its new doubled value matches the block directly below it.

- The algorithm checks the node directly down from the newly doubled block.
- If the values match, the *bottom* node's value is doubled, and the *top* node is deleted using the ***removeNode*** method, which safely bypasses all 4 of its pointers and reassigns the head if necessary.
- This process calls itself recursively (cascadeMerge(bottomNode)) until no more matches are found downwards.

Complexity**:** O(R)

E. Array-Free Console Rendering (printGrid) To strictly adhere to the rule against arrays, even the console output logic was built using linked-list traversal. The printGrid method iterates through logical coordinates but fetches the data by calling findNode(row, col). This helper method dynamically searches the linked list by locating the column floor and walking up until it finds the matching row, printing [ ] if the coordinate is empty.

Complexity: O(R×C×(R+C))

General overview of time complexities:

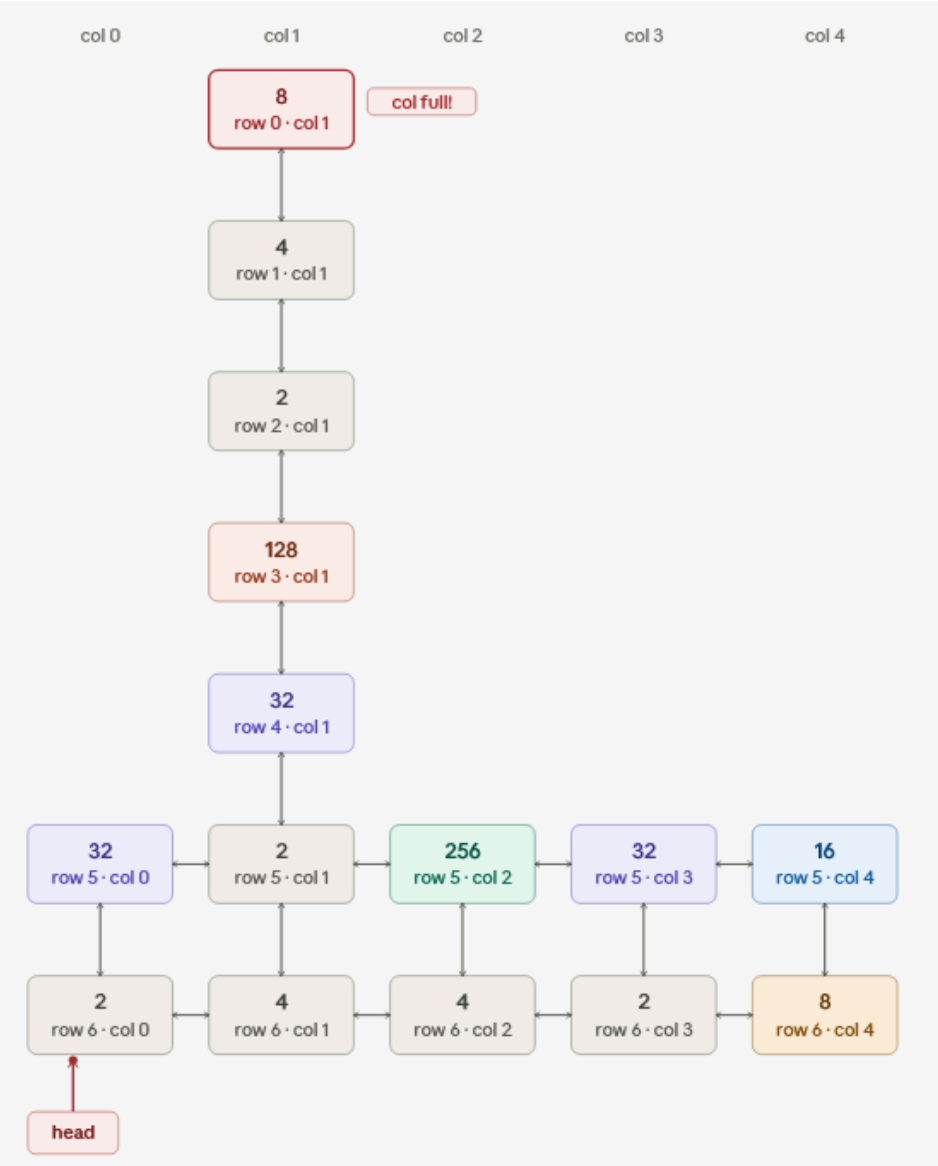
Current complexity is slightly high because "Navigation" starts from head and traverses the structure every time. If we stored an array of "Floor" nodes (e.g., Node[] columnFloors), our getColumnFloor would become **O(1)**, which would reduce the complexity of almost every other method in the class.

4. Game Over Condition

The game actively monitors the height of every column. The isGameOver() method iterates through the columns and calls isColumnFull(col). A column is considered completely full if its peak node reaches row 0 (the very top of the grid). If any column reaches this state, the game terminates.

5. Visual Diagram of the Multi-Linked List

in the final stage:



5. Program Output on Console

initialization:

```
=== Drop Number Game Simulation ===

Step 1: Drop 2 into column 0
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[   2][    ][    ][    ][    ]
-----------------------------------
Step 2: Drop 2 into column 3
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[   2][    ][    ][   2][    ]
-----------------------------------
Step 3: Drop 4 into column 1
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[   2][   4][    ][   2][    ]
-----------------------------------
```

example of cascaded merge:

```
-----------------------------------
Step 28: Drop 8 into column 2
[    ][    ][    ][    ][    ]
[    ][    ][   8][    ][    ]
[    ][   2][  16][    ][    ]
[    ][ 128][  32][    ][    ]
[    ][  32][  64][    ][    ]
[  32][   2][ 128][  32][  16]
[   2][   4][   4][   2][   8]
-----------------------------------
Step 29: Drop 8 into column 2
[    ][    ][    ][    ][    ]
[    ][    ][    ][    ][    ]
[    ][   2][    ][    ][    ]
[    ][ 128][    ][    ][    ]
[    ][  32][    ][    ][    ]
[  32][   2][ 256][  32][  16]
[   2][   4][   4][   2][   8]
-----------------------------------
Step 30: Drop 4 into column 1
[    ][    ][    ][    ][    ]
[    ][   4][    ][    ][    ]
[    ][   2][    ][    ][    ]
[    ][ 128][    ][    ][    ]
[    ][  32][    ][    ][    ]
[  32][   2][ 256][  32][  16]
[   2][   4][   4][   2][   8]
-----------------------------------
```

the final result:

```
Step 31: Drop 8 into column 1
[    ][   8][    ][    ][    ]
[    ][   4][    ][    ][    ]
[    ][   2][    ][    ][    ]
[    ][ 128][    ][    ][    ]
[    ][  32][    ][    ][    ]
[  32][   2][ 256][  32][  16]
[   2][   4][   4][   2][   8]
-----------------------------------
_________game___is___over_________
-------------------------------------------------
BUILD SUCCESS
-------------------------------------------------
Total time:  17.341 s
Finished at: 2026-04-15T13:36:23+03:00
-------------------------------------------------
```

6. GUI output

Drop Number Game — Roaa Alsham
Step: 31 / 31
GAME OVER — column full!
Col 0
Col 1
Col 2
Col 3
Col 4
8
4
2
128
32
32
2
256
32
16
2
4
4
2
8
Drop Next
Auto Play
Reset